

Vulnerability Handling Starter Pack — the files and habits a software team needs first

- [01 - security.txt](#)
 - [What it is](#)
 - [The fields, one line each](#)
 - [Where it goes](#)
 - [Worked example](#)
 - [Common mistakes](#)
 - [Verify it is served correctly](#)
- [02 - Vulnerability Disclosure Policy](#)
 - [What it is for](#)
 - [The template](#)
 - [Why each section exists, and what happens if you drop it](#)
 - [Setting the numbers](#)
 - [Where you genuinely need a lawyer](#)
- [03 - Intake and Triage](#)
 - [The three questions](#)
 - [Where reports arrive](#)
 - [Who reads them](#)
 - [The first hour](#)
 - [Reproducing and confirming](#)
 - [Is it exploitable in your product?](#)
 - [Is it being used right now?](#)
 - [Deciding severity](#)
 - [Triage worksheet](#)
 - [Where this stops and regulation starts](#)
- [04 - Advisory and CVE](#)
 - [When a CVE is appropriate](#)
 - [Two routes to an id](#)
 - [What a good advisory contains](#)
 - [Getting the affected-versions range right](#)
 - [Coordinating timing with the reporter](#)
 - [Advisory template](#)
 - [After publication](#)
- [05 - The Five Files](#)
 - [The list](#)

- 1. SECURITY.md
- 2 and 3. security.txt and the disclosure policy
- 4. The dated decision log
- 5. Support and end-of-life statement
- Closing

```
=====
===== THE VULNERABILITY HANDLING STARTER PACK Toledo Technologies LLC
- Version 1.0
=====
=====
```

NOT LEGAL ADVICE

Written by engineers, for engineers. Not legal advice, not a certification, not a compliance determination. Buying it makes you compliant with nothing, and nothing here creates a lawyer-client relationship. Two parts carry real legal weight and should be read by a lawyer before you publish them: the safe-harbour wording in the disclosure policy, and any promise about response times. Both are marked. Where something depends on where you are incorporated or where your users are, this pack says so instead of guessing.

WHAT THIS IS

Six files covering the vulnerability-handling setup a software team needs before any regulator, customer, auditor, or researcher comes asking. Deliberately small. An afternoon's work, then done.

README.txt this file 01-SECURITY-TXT.md machine-readable contact point 02-VULNERABILITY-DISCLOSURE-POLICY.md the public policy 03-INTAKE-AND-TRIAGE.md what to do when a report lands 04-ADVISORY-AND-CVE.md publishing a fix, getting a CVE 05-THE-FIVE-FILES.md the closing checklist

Every template is meant to be copied, pasted, and edited. Placeholders are ALL CAPS in square brackets, so searching a repo for "[" plus "]" finds everything still to fill in.

WHO IT IS FOR

A team of two to thirty people shipping software - SaaS, a mobile app, an on-prem product, a library, firmware - with no written answer to "where does a stranger send us a security bug?"

Regulation-agnostic on purpose. A four-person team in Kansas with no European customers needs every file here for ordinary reasons: enterprise questionnaires ask for a disclosure policy, researchers who cannot find a contact address sometimes post to social media instead, and a report sitting in a personal inbox during someone's holiday is a report you did not receive.

WHAT THIS IS NOT

Not a bug bounty. No money changes hands anywhere here.

Not a pentest or audit. It covers what to do with findings, not how to find them.

Not a secure development lifecycle, threat model, patch policy, or breach response plan. A vulnerability in your product and an intrusion into your network are different events with different playbooks. This is the first. RFC 9116 draws the same line.

Not a compliance package. No reporting deadlines, no regulator templates, no scope analysis.

WHEN YOU SHOULD BUY THE LARGER KITS INSTEAD

Be honest with yourself about this paragraph. If you sell, ship, or make available a product with a digital element to anyone in the European Union, then Regulation (EU) 2024/2847 - the Cyber Resilience Act - is not a future problem for you: its Article 14 reporting obligations became enforceable on 11 September 2026, the trigger is an actively exploited vulnerability rather than a severe one, and the deadlines cascade at 24 hours, 72 hours, and 14 days to a coordinating national CSIRT and ENISA. This pack gets you to the point of knowing a vulnerability is being exploited. It will not tell you who to notify, on what clock, in what format, or who owns that clock, and that is exactly the separate CRA kit. Likewise, if your concern is liability for defective software rather than reporting, Directive (EU) 2024/2853 applies to products placed on the market after 8 December 2026 and brings its own evidence expectations; the separate PLD kit covers scope, the shift from the 1985 regime, and the update-supportability evidence practice, none of which are here. If either describes you, this pack is a good first step but not the thing you need, and you should buy the relevant kit. If neither does, this pack is the whole job.

Primary sources, so you can check rather than trust us: CRA

<https://eur-lex.europa.eu/eli/reg/2024/2847/oj> PLD

<http://data.europa.eu/eli/dir/2024/2853/corrigendum/2026-05-07/oj>

The PLD date is worth a note. The directive as published said 9 December 2026 in Article

2(1). Corrigendum 2026/90364, linked above, corrected it to 8 December. If you see the older date quoted elsewhere, that is why.

ORDER OF WORK

Day one, about two hours: 1. Create a security@ alias at least two people read. 2. Publish security.txt (file 01) 3. Publish the disclosure policy (file 02) 4. Add SECURITY.md to the repo (file 05)

Before you need it, about one hour: 5. Walk the team through intake and triage (file 03) 6. Read the advisory and CVE process once, so it is not new on the day you need it (file 04) 7. Start the decision log (file 05). Today. Empty is fine.

The decision log is the one people skip. File 05 explains why it is also the one that matters most later.

LICENCE

Use this inside your own organisation and in your own products. Copy the templates into your repos, docs, and customer-facing pages, edit freely, ship them. Do not resell or redistribute the pack itself as a product.

If something here is wrong, we would rather know.

Toledo Technologies LLC

01 - security.txt

NOT LEGAL ADVICE. Technical guidance about a file format. Not legal advice, not a certification, not a compliance determination.

What it is

A plain text file at a fixed path telling a stranger who found a bug in your product where to send it. Format: **RFC 9116** (April 2022). <https://www.rfc-editor.org/rfc/rfc9116.html>

Without it a researcher guesses. They try security@, which bounces. They try the contact form, which goes to sales. They open a public GitHub issue with a working exploit in it, because that was the only channel they could find. That last outcome is what this file prevents.

RFC 9116 is explicit that this is for **vulnerability response** - flaws in your product - not **incident response**, which is what you do when your own systems are broken into.

The fields, one line each

Only two are mandatory.

Field	Required?	Meaning
Contact	Yes, 1+	Where to send the report. mailto:, https:, or tel:. Listed several times, the order indicates priority.
Expires	Yes, exactly 1	Date/time after which the file should not be trusted. RFC 3339 format. RFC 9116 recommends under a year out.
Encryption	No	URI where your public key lives. The key itself must not be pasted in - only a link, a dns: URI, or an openpgp4fpr: fingerprint.
Acknowledgments	No	Link to the page thanking reporters. Must be https://.

Field	Required?	Meaning
Preferred-Languages	No, max 1	Comma-separated RFC 5646 tags. No priority ordering. Absent means English is assumed.
Canonical	No, but publish it	Where this file is meant to live. See mistakes below.
Policy	No, but publish it	Link to your disclosure policy. Saves you the most email.
Hiring	No	Security job openings. Skip if not hiring.
CSAF	No	Link to a CSAF provider-metadata.json. Almost no small team has one. Skip.

Format rules people trip on: plain text, UTF-8, served as Content-Type: text/plain; charset=utf-8; one field per line, ending LF or CRLF; no chaining multiple values onto one field; # starts a comment; field names are case-insensitive; **HTTPS only**.

Where it goes

```
https://yourdomain.example/.well-known/security.txt
```

Required by RFC 9116, built on the well-known URI registry in RFC 8615. You may also keep a copy at /security.txt or redirect from there for older tooling; if both exist, /.well-known/ wins.

Scope is narrower than people assume. RFC 9116 section 3.1: the file applies only to the exact domain or IP in the URI used to fetch it - not subdomains, not the parent domain. A file at example.com says nothing about app.example.com. It may, however, cover the products and services your organisation provides, which is the case that matters for most software teams and which you should spell out in the linked policy.

Worked example

Invented for this walkthrough, using the .example TLD that RFC 2606 reserves for documentation, so nothing here points at a real organisation.

Larkfield Systems. Six people, one self-hosted inventory product plus a small hosted dashboard, no security staff, and one engineer - Priya - who owns this by default.

```
# security.txt for larkfield.example
# Covers this domain and the Larkfield Stock product.
# Questions about the policy itself: hello@larkfield.example

Contact: mailto:security@larkfield.example
Contact: https://larkfield.example/security/report
Expires: 2027-06-30T23:59:00Z
Encryption: https://larkfield.example/security/pubkey.asc
Policy: https://larkfield.example/security/disclosure-policy
Acknowledgments: https://larkfield.example/security/thanks
Preferred-Languages: en
Canonical: https://larkfield.example/.well-known/security.txt
```

Note what they did. security@ is a **distribution list with three people on it**, not a mailbox, listed first because email is what they actually want. The web form is second, for researchers who will not send email. Expires is nine months out with a reminder at seven. The key sits on their own domain, not a keyserver they do not control. Canonical matches the URL exactly, including the missing www..

Common mistakes

An Expires date in the past. The most common failure, and worse than having no file. RFC 9116 section 5.3 says so directly. An expired file tells a researcher nobody is minding this, so the address probably bounces, so why bother. Put renewal in a shared calendar or in CI.

No Canonical. An attacker with write access to your site can replace or redirect this file and quietly collect your vulnerability reports. Canonical lets a researcher check the file claims to live where they found it; if it lists canonical URIs and theirs is not among them, RFC 9116 tells them not to trust it. It is also what makes a signature meaningful, by binding signed content to a location.

A personal email address. priya@ works until Priya is on a plane, or leaves. Use a role address at least two people read, and test it from outside today. A security@ alias forwarding to an inbox nobody opens is the same failure with extra steps.

Pointing a field into /.well-known/. Non-obvious. RFC 9116 section 6: referenced resources must not point into the well-known namespace unless registered with IANA, so Encryption: https://example.com/.well-known/pubkey.asc is invalid. Host keys at an ordinary

path. Canonical is the exception, because security.txt itself is registered.

Pasting the PGP key into the file. Forbidden. Encryption takes a URI, not key material.

Using http://, or serving it as HTML. Web URIs in Acknowledgments, Canonical, Encryption, and Policy must begin with https://. If your framework returns text/html, or a 200-with-a-404-page, parsers break.

Signing it then letting the signature rot. Signing is recommended, not required. An unsigned, in-date, accurate file beats a signed one you forgot to re-sign after editing.

Verify it is served correctly

From a machine outside your network:

```
# 1. Status, content type, redirects
curl -sSIL https://larkfield.example/.well-known/security.txt
# Want: HTTP/2 200 and content-type: text/plain; charset=utf-8

# 2. The body
curl -sSL https://larkfield.example/.well-known/security.txt

# 3. Confirm Expires is still in the future (GNU date)
curl -sSL https://larkfield.example/.well-known/security.txt \
| awk -F': ' 'tolower($1)=="expires"{print $2}' \
| xargs -I{} date -d {} +%s \
| awk -v now="$(date +%s)" '{print ($1 > now) ? "OK" : "EXPIRED"}'

# 4. Check the legacy path is not serving a different, older file
curl -sSIL https://larkfield.example/security.txt
```

Then the human checks no parser does for you: send a real test email to the Contact address from an outside account and confirm a human saw it; click the Policy link; fetch the Encryption URI and confirm someone still holds the private half.

The validator at <https://securitytxt.org/> flags format errors. Treat it as a linter, not proof - it cannot tell you whether anyone reads the inbox. **Put steps 1 and 3 in CI weekly**, and this becomes a thing that tells you when it breaks rather than a thing you forget.

02 - Vulnerability Disclosure Policy

NOT LEGAL ADVICE. A template for a public document that makes commitments on your organisation's behalf. Not legal advice, not a certification, not a compliance determination. **The safe-harbour section must be reviewed by a lawyer qualified in your jurisdiction before you publish it** - see the last section.

What it is for

The page your security.txt Policy: field points at. It answers four questions a researcher has before spending an evening on your product: am I allowed to test this and will you sue me; what is in scope; will a human reply and when; can I talk about it afterwards.

If you do not answer these, some researchers still report. The careful ones - usually the ones finding the serious things - often will not, because reporting to a company with no stated policy carries legal exposure that is easy to underestimate from the inside.

This is a disclosure policy, not a bug bounty. There is no money in this template. Money brings tax questions, sanctions-screening questions, and a large increase in low-quality submissions. Do not add it casually.

The template

Copy the block. Replace every [PLACEHOLDER]. Delete the >> guidance lines.

```
# Vulnerability Disclosure Policy

**Last updated: [YYYY-MM-DD]**

## Our commitment

[COMPANY LEGAL NAME] takes security reports seriously. If you have
found a security issue in our products or services, we want to hear
about it, and we will work with you to understand and fix it.

We do not operate a paid bug bounty programme. We do offer public
credit to reporters who want it.
```

How to report

Email: [security@yourdomain.example]

Web form: [https://yourdomain.example/security/report]

PGP key: [https://yourdomain.example/security/pubkey.asc]

Please include the information listed under "What we ask of you" below. If you believe the issue is being actively exploited against real users right now, say so in the subject line.

Scope

The following are in scope:

- [PRODUCT NAME] versions [X.Y] and later
- The web application at [https://app.yourdomain.example]
- The API at [https://api.yourdomain.example]
- Our published packages: [npm: @org/package], [PyPI: package-name]
- The marketing site at [https://yourdomain.example]

If it is ours and it is not listed as out of scope below, report it anyway. We would rather decline a report than miss one.

Out of scope

These are not in scope, and reports about them will usually be closed without detailed analysis:

- Systems and services operated by third parties on our behalf, including [LIST YOUR MAIN VENDORS - e.g. our payment processor, our email provider, our CDN]. Report those to the vendor directly.
- Findings that require physical access to a user's unlocked device.
- Findings that require a fully compromised or rooted device, or a malicious browser extension already installed.
- Reports from automated scanners with no demonstrated impact.
- Missing security headers, cookie flags, or TLS configuration choices with no demonstrated exploit path.
- SPF, DKIM, and DMARC configuration reports, unless you can demonstrate a working spoof that reaches an inbox.
- Denial of service, volumetric or otherwise. Please do not test for this. See prohibited activity below.
- Rate limiting and brute force on endpoints where we have documented that behaviour as accepted.
- Social engineering of our staff, customers, or vendors.
- Self-XSS, clickjacking on pages with no sensitive action, and issues requiring an unrealistic degree of user interaction.
- Vulnerabilities in dependencies where we are not the maintainer and no exploit path exists through our product. Report those upstream; tell us as well if our users are affected.

- End-of-life versions listed in our support statement at [https://yourdomain.example/support-policy].

>> Edit this list. It should describe your product, not ours.
>> A generic out-of-scope list is a sign you have not thought
>> about your own attack surface.

Prohibited activity

While testing, you must not:

- Access, modify, delete, or store data belonging to anyone other than yourself or a test account you control.
- Degrade, interrupt, or deny service to our systems or our users.
- Run load, stress, or automated scanning at volume against our production systems.
- Use a finding to pivot further into our infrastructure than is necessary to demonstrate the issue.
- Publicly disclose the issue before the timeline described below.
- Demand payment in exchange for withholding details of a finding.

If you accidentally access data that is not yours, stop, do not save a copy, and tell us in your report what you saw. Doing this will not count against you.

Safe harbour

>> =====
>> LEGAL REVIEW REQUIRED BEFORE YOU PUBLISH THIS
>> SECTION. Read the notes in file 02 first.
>> =====

If you make a good faith effort to comply with this policy during your research, we will consider your research to be authorised, we will work with you to understand and resolve the issue quickly, and we will not recommend or pursue legal action against you in connection with your research.

If a third party brings legal action against you in connection with research you carried out in compliance with this policy, we will make it known that your activity was conducted in compliance with this policy.

This authorisation is limited to conduct that complies with this policy. It does not extend to activity prohibited above, and it cannot waive the rights of any third party.

If you are unsure whether a particular action is permitted, ask us at [security@yourdomain.example] before you do it. We will answer.

What we ask of you

Please include:

- A description of the issue and why you think it is a security problem.
- The exact product, version, or URL affected.
- Step by step instructions to reproduce it. A short video or a minimal proof of concept helps a great deal.
- What an attacker could achieve, in your view.
- Whether anyone else has this information.
- How you would like to be credited, or that you prefer not to be.

Please use a test account where you can. Please do not include third-party personal data in your report.

What you can expect from us

- **Acknowledgement within [2] business days.** A human, not an autoresponder, confirming we have it.
- **An initial assessment within [10] business days**, telling you whether we have reproduced it, whether we consider it in scope, and our initial view of severity.
- **Regular updates at least every [14] days** while we work on it.
- **Notice before we publish** any advisory that names or describes your finding, and a chance to review the wording.
- **Credit** in the advisory and on our acknowledgements page, in the form you asked for, unless you asked for none.

We aim to have a fix available within [90] days of confirming a valid report. Some issues take longer. If that happens, we will tell you why rather than going quiet.

Coordinated disclosure

We ask that you give us [90] days from your initial report before disclosing publicly, so that users have a fix to move to.

If we fix the issue sooner, we will publish sooner, and you are free to publish once our advisory is out.

If we have not fixed it within [90] days, we would like to talk to you about the timeline rather than have you discover our silence. We will not ask you to stay quiet indefinitely, and we will not treat publication after an agreed deadline as a breach of this policy.

If the issue is being actively exploited in the wild, both of us may

```
need to move faster than this timeline. We will tell you if we
believe that is the case.
```

```
## Changes to this policy
```

```
We will update this page when our process changes, and change the
"last updated" date at the top. The current version always lives at
[https://yourdomain.example/security/disclosure-policy].
```

```
## Contact for questions about this policy
```

```
[security@yourdomain.example]
```

Why each section exists, and what happens if you drop it

Our commitment says “bounty” out loud. Drop it and much of your inbound becomes “what is the payout?”

How to report is the operative sentence of the page. Drop it and the policy is decorative. Keep it above the fold.

Scope points researchers at what you care about, raising the quality of what arrives. The “report it anyway” line matters: a strict allowlist means the genuinely novel finding that fits no bullet gets dropped. Drop the section and you get reports about domains you do not own.

Out of scope is the section that buys back your weekends. Most of what arrives at a new security@ is scanner output about missing headers; naming those categories lets you close them in one line with a link. Drop it and every low-value report becomes a negotiation - and the negotiations are what burn out whoever does triage.

Prohibited activity does two jobs: it protects your users (the line about not touching other people’s data is the important one), and it is the fence that safe harbour points back to when it says “conduct that complies with this policy.” Drop it and your safe-harbour clause has nothing bounding it, which is your problem, not the researcher’s.

Safe harbour is why careful researchers report rather than staying quiet. Testing software without permission can carry liability under computer-misuse law in many jurisdictions; published authorisation turns unauthorised access into authorised access. Drop it and you still get reports, but fewer from people with something to lose - which skews against experienced professionals and towards anonymous disclosure.

What we ask of you cuts round trips from four to one. “Does anyone else have this?” is the highest-value question on the list, almost nobody asks it, and the answer changes your timeline completely. Drop it and you receive reports reading, in full, “your site is vulnerable to XSS.”

What you can expect from us addresses the real failure in disclosure, which is not refusing to fix something but going silent - and silence is what makes a patient researcher publish early.

Coordinated disclosure sets timing expectations before anyone is under pressure. Note the template asks rather than claiming a veto: you cannot stop someone publishing, and pretending otherwise reads as hostile to exactly this audience. Drop it and every report becomes a fresh timing negotiation conducted while you are shipping a patch.

Last updated lets a reporter cite the version they relied on.

Setting the numbers

Pick numbers you will hit. A missed 24-hour promise does more damage than an honest 5-day one.

Commitment	Conservative	Ambitious
Acknowledge receipt	5 business days	1 business day
Initial assessment	15 business days	5 business days
Status updates	every 30 days	every 14 days
Target fix	120 days	60 days
Disclosure window	90 days	90 days

Ninety days is the de facto industry default and researchers rarely argue with it. If you promise faster than the conservative column, make sure someone is on the hook when your one security-literate engineer is on holiday.

Where you genuinely need a lawyer

Four points, no bluffing.

1. The safe-harbour wording. Whether a private authorisation actually protects a researcher from criminal or civil liability is a question of law in each jurisdiction and it varies - in the US it interacts with the Computer Fraud and Abuse Act and state computer-crime statutes, in the UK with the Computer Misuse Act 1990, elsewhere with local equivalents. A company promising not to sue does not bind a public prosecutor. The template uses the careful, widely-used construction (“we will consider your research authorised”, “we will not pursue legal action”) rather than the overreaching “you will not be prosecuted,” which nobody can promise. Have your lawyer read it: fifteen minutes of their time, and the one paragraph here where wrong wording creates real exposure for you and for a researcher relying on it.

2. Supporting a researcher against third-party legal action. That paragraph is a commitment. Ask how far you want to go and whether your insurer cares.

3. Response-time commitments. In most places a published policy is not a contract; in some contexts, particularly if repeated in a customer agreement or enterprise security addendum, it can become one. If you have enterprise customers, check these numbers against what you signed.

4. Anything involving personal data. If a report involves user data the researcher accessed, you may have breach-notification duties - GDPR in the EU and UK, state breach-notification laws in the US, among others. Those are triggered by the access, not by your policy. Decide in advance who makes that call.

You do **not** need a lawyer for the rest of the page. Do not let a pending review of one paragraph stop you shipping the other eight sections this week. Publish with safe harbour marked as under review, or without it and add it later. A policy without safe harbour beats no policy at all.

03 - Intake and Triage

NOT LEGAL ADVICE. An operational runbook. Not legal advice, not a certification, not a compliance determination. It covers how to work out what you are looking at, not who you must notify or on what clock - see the closing note.

The three questions

A report arrives. Within about an hour you need to know:

1. **Is it real?** Can you reproduce it.
 2. **Does it matter in *your* product?** Not in the abstract - in the code you ship and the configuration you run.
 3. **Is anyone using it right now?** A different question from the first two, and the one most teams never explicitly ask. It gets the most space here because it is the one with consequences.
-

Where reports arrive

You have more inboxes than you think. List yours honestly:

Channel	Watched by	Realistic latency
security@ alias	[NAMES]	[X] hours
Web report form	[NAMES]	[X] hours
GitHub private vulnerability reporting	repo admins	until someone looks
Public GitHub issue	issue triager	hours to days
Support ticket queue	support	hours
A DM to a founder	one person	unbounded
A conference hallway	one person	unbounded

Channel **Watched by** **Realistic latency**

A CVE in a dependency you use usually nobody **usually never**

The last three are the leaks. Two rules fix most of it.

Anything resembling a security report goes to security@ immediately, unopened if necessary. Support staff should route security reports, not triage them. Say so explicitly; they will not guess.

A public issue containing a working exploit gets handled, not debated. Decide the order now: most teams edit or delete the issue body, reply publicly that they are moving to a private channel, then email the reporter. You do not want to improvise that in a repo people are watching.

If you have a public repo, turn on GitHub's private vulnerability reporting. Free for public repositories, it gives reporters a channel that does not depend on email and puts the report where you will later draft the advisory. <https://docs.github.com/en/code-security>

Who reads them

Name two people. Not a team, not a rota - two named humans, one primary this month. The failure mode is never "nobody was assigned," it is "each assumed the other had it."

Security intake

Primary this month:	[NAME]	[phone]
Backup:	[NAME]	[phone]
Escalation:	[NAME - can authorise an unscheduled release]	
Reviewed:	[YYYY-MM-DD]	

The escalation contact matters because triage sometimes ends in "we ship today," and whoever is triaging at 22:00 should not have to work out who can say yes.

The first hour

In order. Resist the urge to start fixing.

0-5 min. Acknowledge. Two sentences: we have it, a human is looking, you will hear from us by [date]. Highest-return action in this file. It stops the clock in the reporter's head - the one otherwise running towards "these people are ignoring me."

5-10 min. Open a private tracking item. A draft GitHub security advisory, or a ticket not everyone can read. Not a public issue. Assign an internal id now (SEC-2026-001) and use it in every message afterwards.

10-20 min. Read it and classify the channel. Your code, a dependency, or a service you merely buy? Three different destinations. A flaw in third-party SaaS is not yours to fix - forward it and tell the reporter you did.

20-40 min. Attempt to reproduce.

40-50 min. Ask the exploitation question, even if reproduction is unfinished. It is fast and it can change the next twenty-four hours.

50-60 min. Decide and record. One of: confirmed / not reproducible / out of scope / needs more from reporter / needs specialist help. Write the decision, the reason, and the date in the tracking item. Reply to the reporter.

Do not skip the write-down because the answer feels obvious. In two years, "not exploitable because the endpoint requires an authenticated admin session" is defensible. "We must have looked at it" is not.

Reproducing and confirming

Work from a clean environment, not your dev machine with its eight local overrides.

1. **Pin the version.** Exact release or commit. If the reporter did not say, ask - do not assume `main` is what users run.
2. **Match the configuration,** and note whether it was default. That distinction matters enormously in the advisory.
3. **Follow the reporter's steps exactly on the first pass.** If it fails, then vary.
4. **If it does not reproduce, say so specifically.** Not "cannot reproduce" but "tried 2.4.1 and 2.5.0, default config, PostgreSQL 16, step 3 returned 403." Reporters usually close that gap in one reply. A bare "cannot reproduce" reads as a brush-off and often ends the conversation with the bug still present.
5. **Find the root cause, not the symptom.** Before sizing the fix, find the function that is actually wrong and ask what else reaches it. Symptom fixes come back.

6. **Check older supported versions.** Almost always affected, almost always forgotten until after the release.
 7. **Write down what you did** - commands, versions, outcome. You need it for the advisory and you will not remember it.
-

Is it exploitable in your product?

A finding can be real and still not be a vulnerability in what you ship. Ask, in order:

- **Is the vulnerable code path reachable?** A flaw in a library function you never call is not exploitable through your product. Check rather than assuming either way.
- **Who can reach it?** Anonymous internet / any authenticated user / an administrator / someone already inside. Each step is roughly an order of magnitude of real risk.
- **What preconditions are required?** A non-default setting, a specific topology, a feature flag. If a precondition is rare the impact is smaller - but check telemetry rather than guessing what customers enable.
- **What does an attacker get?** Read data, write data, execute code, escalate privilege, deny service. “Execute code” and “read other tenants’ data” are the two that wake people up.
- **Is there a mitigation available today?** A config change, a proxy rule, disabling a feature. If yes, you can tell customers something useful hours before you have a patch - often the most valuable output of the whole triage.

Two notes on dependency reports. **OSV.dev** (<https://api.osv.dev>) is a public, no-key API for checking whether a package version is affected, with one sharp edge: `/v1/querybatch` returns only `{id, modified}` per match, with **no aliases**, so a GHSA id from a batch query must be resolved through `/v1/vulns/{id}` before you can see its CVE alias. If you script this, that second call is not optional, and skipping it is the usual reason such scripts silently miss things. And **reachability beats version matching** - a scanner reporting that `libfoo 1.2.3` has a CVE is the start of triage, not the end.

Is it being used right now?

The question small teams systematically fail to ask. Five minutes on every confirmed report.

“Actively exploited” means someone is using this against real systems now - not that it is severe, not that a proof of concept exists. A critical vulnerability nobody has touched and a medium one being used in the wild are different situations, and the second is more urgent.

Signals, cheapest first:

1. Ask the reporter. “Did you find this in the wild, or in the code?” People who find bugs during incident response answer very differently from people reading source. One line of email.

2. Search your own logs for the pattern you just reproduced, across whatever window you retain. Hits before the report date are the strongest signal you will get, and the moment to stop and get help. This only works if retention is long enough - if yours is seven days, note that gap now, while fixing it is cheap.

3. Check the CISA Known Exploited Vulnerabilities catalogue. Public JSON, no key:

```
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json
```

As of catalogue version 2026.09.14 it held 1,710 entries. It skews heavily towards widely-deployed software, so a small vendor’s own product rarely appears - but your **dependencies** will, and that is the useful case. A weekly job diffing KEV against your dependency manifest is about thirty lines of code and is the highest-value monitoring most small teams are not doing.

4. Watch the softer signals. A scanning spike against that endpoint. Several unrelated customers reporting odd behaviour in the same component. Your finding appearing in a public exploit repository. Any of these move you from “we think not” to “we do not know,” a materially different answer.

Record the conclusion either way, with the date and the basis. “Application logs searched 2026-09-15, 90-day retention, no matches” is worth writing down. The absence of that note is indistinguishable from never having looked.

Deciding severity

Use CVSS if you want a number for an advisory - v4.0 is current, v3.1 still widely understood, either is fine as long as you publish the full vector string and not a bare score. <https://www.first.org/cvss/>

But know what it is: a portable description of a vulnerability in the abstract. It is not your risk and it is not a decision. A 9.8 in a code path your product cannot reach is not a 9.8 for you. For the decision itself, a plain internal scale serves a small team better:

Level	Meaning	Response
Critical	RCE, authentication bypass, or cross-tenant data access without unusual preconditions - or anything you believe is actively exploited	Drop other work. Out-of-band release. Notify customers.
High	Serious impact, but needs authentication, a non-default configuration, or user interaction	Fix in days. Expedited patch release.
Medium	Real but limited impact, or high impact behind significant preconditions	Next scheduled release.
Low	Defence in depth, or impact requiring an attacker who already has what they would be after	Backlog, with a date attached.

Note the clause in the Critical row. Active exploitation promotes severity regardless of score. That is the most useful rule in the table, and the one that matters if a regulator is ever involved.

If you want something more structured, CISA's SSVC decision trees are free and built for "what do we do" rather than "how bad is this in general." <https://www.cisa.gov/ssvc>

Triage worksheet

Copy into your private tracking item and fill in as you go. Short on purpose - a worksheet nobody completes is worse than none.

TRIAGE WORKSHEET

Internal ID: SEC-[YYYY]-[NNN]
Date received: [YYYY-MM-DD HH:MM TZ]
Received via: [email / form / GitHub / support / other]
Reporter: [name or handle, or "anonymous"]
Acknowledged at: [YYYY-MM-DD HH:MM TZ] by [NAME]
Triage owner: [NAME]

--- 1. What was reported -----

Summary (one sentence):

Component / product:

Versions claimed affected:

--- 2. Reproduction -----

Reproduced? [yes / no / partial / not yet]

Environment used: version [], config [default / non-default:]

Steps confirmed: [as reported / modified - how:]

Root cause: [file / function / commit, or "unknown"]

Other versions checked:

Notes:

--- 3. Exploitability in OUR product -----

Vulnerable path reachable? [yes / no / unclear]

Who can reach it? [anonymous / any user /
admin / internal only]

Preconditions required:

Attacker gains: [RCE / data read / data write /
privilege escalation / DoS /
other:]

Mitigation available today? [none / describe:]

--- 4. Active exploitation -----

Reporter found it in the wild? [yes / no / did not answer]

Own logs checked? [yes / no]

Retention window searched: [e.g. 90 days]

Result: [no matches / matches - detail:]

CISA KEV checked (deps)? [yes / no / n/a]

Other signals: [scanning spike / public PoC /
multiple customer reports / none]

CONCLUSION: [no evidence of exploitation /
unknown / evidence found]

Basis and date: [e.g. "logs searched 2026-09-15,
90-day window, no matches"]

--- 5. Severity -----

Internal level: [critical / high / medium / low]

CVSS vector: [optional - include the full vector string]

Reasoning:

--- 6. Decision -----

Decision: [fix now / fix next release / backlog /
not a vulnerability / out of scope /
upstream - forwarded to:]

Decided by: [NAME] Date: [YYYY-MM-DD]

Reason (2 lines, written for someone reading this in 2 years):

Advisory needed? [yes / no] CVE needed? [yes / no]

Reporter informed of decision? [yes - date: / no]

Target fix date:	[YYYY-MM-DD]
Next review date:	[YYYY-MM-DD]

Section 4 and the reason line in section 6 are the two people skip, and the two with value after the immediate work is done. Fill them in even when the answer is boring.

Where this stops and regulation starts

This runbook gets you to a defensible answer on one question: **do we have evidence that this vulnerability is being actively exploited?**

That is not academic. Under the EU Cyber Resilience Act - Regulation (EU) 2024/2847 - active exploitation, not severity, triggers the Article 14 reporting obligations, enforceable since 11 September 2026, running on a cascade of 24-hour, 72-hour and 14-day deadlines to a coordinating national CSIRT and to ENISA. Primary source: <https://eur-lex.europa.eu/eli/reg/2024/2847/oj>

That is deliberately all this pack says about it. **If the CRA applies to you, the deadline mechanics, who notifies whom, what goes in each report, and how you rehearse it are substantial work covered in the separate CRA kit.** What this file gives you is what has to exist first: a process that can answer the exploitation question, and a written record of when you asked and what you found. Without that, the reporting obligations have nothing to operate on.

If the CRA does not apply to you, nothing changes. The exploitation question is the right question for its own sake.

04 - Advisory and CVE

NOT LEGAL ADVICE. This describes public processes run by the CVE Program and by GitHub. Not legal advice, not a certification, not a compliance determination. Requesting a CVE is not a regulatory filing and satisfies no reporting obligation.

When a CVE is appropriate

Request one when all three hold: the vulnerability is in software **other people run or depend on**; there is a **version boundary that matters**, so a user needs to know which side they are on; and the affected code is **publicly available or commercially distributed**.

Skip it when the flaw is only in **your own hosted service** and you fixed it in production - nobody has a version to upgrade, so tell affected customers directly. Skip it for **unreleased code**, for **hardening with no exploit path** (calling that a vulnerability devalues the ones that are), and for bugs in **someone else's dependency** where you are not the maintainer.

A CVE id is a coordination mechanism: a stable name so a scanner, a customer's compliance team, and a distro packager can discuss the same thing. If nobody needs to coordinate, it adds nothing. Note the asymmetry - an advisory without a CVE is fine and common; a CVE without an advisory is close to useless.

Two routes to an id

Route A - GitHub, if your code is there

For a small team with a public repo this is the path, and it is free.

1. Repo > **Security > Advisories > New draft security advisory**.
2. Fill in affected versions, patched versions, severity, description, CWE, credit.
3. Optionally open a **temporary private fork** to develop the fix out of sight.
4. Click **Request CVE**. GitHub is a CNA and can assign an id for code it hosts.
5. Ship the fix, then click **Publish advisory**.

Docs: <https://docs.github.com/en/code-security>

Three things to know first. **Add the fixed version before publishing** - GitHub warns about this and it is right, because publishing without one makes Dependabot alert your users to a problem while offering nowhere to go. **Publishing deletes the temporary private fork**, so merge your work back first. And **request the CVE before publishing**, which is the order the flow expects.

Advisories for an ecosystem the dependency graph supports propagate into the GitHub Advisory Database, which is where many scanners actually read from. For most projects that propagation is worth more than the CVE id itself.

Route B - the CNA route, for everything else

CVE ids are assigned by CVE Numbering Authorities, each with a defined scope. The rule: **find the CNA whose scope covers the affected product**.

1. Search the partner list at <https://www.cve.org>. If a CVE exists for this product already, its "Assigning CNA" field names who to ask.
2. Read that CNA's disclosure policy, then contact them by their published method.
3. If no CNA covers it, use a **CNA of Last Resort**: MITRE for general and open-source product vulnerabilities not covered by a listed CNA, or CISA ICS for industrial control systems and medical devices.

Official walkthrough for non-CNAs:

<https://www.cve.org/ReportRequest/ReportRequestForNonCNAs>

For a vulnerability in your own product where no CNA covers you, MITRE as CNA-LR is the normal path. It is a web form. Expect days rather than hours, and expect to be asked for a public reference - so have the advisory URL ready.

Should you become a CNA? Almost certainly not yet. It means operating the assignment process yourself, indefinitely, under the Program's rules. That suits a vendor publishing many advisories a year; for one or two, GitHub or MITRE costs nothing.

What a good advisory contains

It answers one question for a stressed reader: **am I affected, and what do I do in the next ten minutes?**

1. **A title stating the impact**, not the bug class. “Authentication bypass in the admin API” beats “Improper access control.”
2. **Identifiers** - yours, plus CVE and GHSA if they exist.
3. **Affected and fixed versions**, at the top.
4. **Impact in two or three sentences**: what an attacker can do and what they need to start.
5. **How to tell if you are affected** - a version check, a config value, a log pattern. Hugely appreciated and usually missing.
6. **Mitigation** for people who cannot upgrade today.
7. **Credit**, in the form the reporter asked for.
8. **A timeline** with dates.
9. **A severity score with the full vector string**, not a bare number.

Leave out working exploit code, internal ticket numbers, blame, and hedging. “We take security seriously” reads as filler to exactly this audience.

Getting the affected-versions range right

More advisories are wrong here than anywhere else, and this field is machine-read by the scanners your users depend on. Wrong means someone stays vulnerable while believing they are fine.

State both ends. “Versions before 2.5.1” is ambiguous - does it include 1.x? Does it include 2.5.0-rc1? Write `>= 2.0.0, < 2.5.1`.

Handle every maintained branch separately. If you backported to 1.9 and 2.4 as well as fixing 2.5:

```
Affected:  >= 1.0.0, < 1.9.7
           >= 2.0.0, < 2.4.12
           >= 2.5.0, < 2.5.1
Fixed in:  1.9.7, 2.4.12, 2.5.1
```

A user on 2.4.9 who reads only “fixed in 2.5.1” concludes they need a major upgrade they cannot schedule, and does nothing.

Say where it was introduced, if you know. If the bug arrived in 2.3.0 then 2.0-2.2 are unaffected, which saves many people a lot of work. If you do not know, say “all versions before X” rather than guessing a lower bound.

Do not forget pre-releases. Under semver 2.5.0-rc1 sorts before 2.5.0, and ranges that ignore this drop release-candidate users through the gap.

The CVE record format mirrors this. Its affected structure uses `defaultStatus` plus a `versions` list, each entry carrying `version` (lower bound), `status`, `versionType`, and usually `lessThan`:

```
{
  "vendor": "Larkfield Systems",
  "product": "Larkfield Stock",
  "defaultStatus": "unaffected",
  "versions": [
    {
      "version": "2.0.0",
      "lessThan": "2.5.1",
      "versionType": "semver",
      "status": "affected"
    }
  ]
}
```

`defaultStatus: "unaffected"` means “anything unlisted is fine” - right when you know your version history. If you genuinely do not know the status of old releases, “unknown” is honest, and is what the schema defaults to when `defaultStatus` is omitted. Reference: <https://cveproject.github.io/cve-schema/schema/docs>

Sanity check before publishing. Take three real versions - one affected, one not, and the boundary release - and evaluate them against your own range by hand. That catches the off-by-one, which is the error that actually happens.

Coordinating timing with the reporter

The reporter started a clock the moment they emailed you, whether or not they said so. Your job is to make it a shared clock.

Agree a date in writing, early. “Targeting a fix by 14 November and publication on 21 November - does that work?” One message, and most friction never appears.

Send the draft advisory before publishing. They catch a technical error roughly a third of the time and it costs you nothing. Set a review window - 48 hours is normal - so it does not become an open-ended veto. Check the credit line explicitly: name, handle, employer, or nothing. Do not credit someone who asked to stay anonymous.

If you need more time, ask before the deadline, not after. Once, with a specific new date and a real reason. Researchers overwhelmingly grant a first extension asked for honestly. What they do not grant is silence followed by a request on day 89.

If the reporter wants to publish before you are ready, you can ask but you cannot compel. Argue on user harm - “400 installations have no patch to move to” - not on your release schedule. Sometimes you lose, and then the right response is to ship whatever mitigation you have and publish the same day.

If it is already being exploited, the calculus inverts. Users under attack need to know now, even if all you have is a config change and a detection rule. Publish early, mark it provisional, update later.

Never threaten a reporter, invoke a lawyer over publication timing, or quietly ship the fix and hope nobody notices. The last is the most common and most damaging: a silent patch means users do not know to prioritise the upgrade, leaving them exposed while attackers diff the release, and it is the fastest way to guarantee the next researcher goes straight to full public disclosure.

Advisory template

```
# [ADVISORY ID] - [Impact-first title]

| | |
|---|---|
| **Advisory ID** | [LARK-2026-001] |
| **CVE** | [CVE-YYYY-NNNNN or "not assigned"] |
| **GHSA** | [GHSA-xxxx-xxxx-xxxx, if applicable] |
| **Published** | [YYYY-MM-DD] |
| **Last updated** | [YYYY-MM-DD] |
| **Severity** | [Critical / High / Medium / Low] |
| **CVSS** | [score] ([full vector string]) |
| **CWE** | [CWE-NNN: name] |

## Affected versions

| Product | Affected | Fixed in |
|---|---|---|
| [PRODUCT] | >= [X.Y.Z], < [A.B.C] | [A.B.C] |
| [PRODUCT] | >= [P.Q.R], < [S.T.U] | [S.T.U] |

>> One row per maintained branch. State both ends of every range.
```

Versions before [X.Y.Z] are [not affected - the flaw was introduced in [X.Y.Z] / of unknown status and no longer supported].

Summary

[Two or three sentences. What an attacker can do, and what they need in order to do it. No jargon a competent generalist would not recognise.]

Impact

[One paragraph. Concrete consequences: which data, whose data, what level of access. Say plainly if it crosses a tenant boundary or reaches other users' data.]

Exploitation requires [authentication / no authentication / admin privileges / a non-default configuration: [WHICH]].

Am I affected?

- Check your version: `[COMMAND OR UI PATH]`
- [If configuration-dependent:] You are only affected if `[SETTING]` is set to `[VALUE]`. Check with `[COMMAND]`.
- [If there is a detectable signature:] Requests exploiting this appear in logs as `[PATTERN]`.

What to do

****Upgrade to [A.B.C]**** [Link to release / install command]

****If you cannot upgrade immediately:**** [Specific mitigation - config change, block a path at the proxy, disable a feature - or "no mitigation is available; upgrading is the only fix."]

Active exploitation

[We have no evidence that this vulnerability has been exploited.
/ We are aware of exploitation in the wild as of [DATE].]

>> State this either way. "No evidence" is information; an
>> absent section makes readers assume the worst.

Technical detail

[Enough for a security engineer to understand and verify the issue. Root cause, affected code path, why the fix works. No weaponised exploit code.]

Credit

Reported by [NAME / HANDLE][, of [ORGANISATION]]. Thank you.

>> Or "Reported by a researcher who wishes to remain
>> anonymous." Or omit if found internally.

Timeline

Date	Event
--- ---	
[YYYY-MM-DD]	Report received
[YYYY-MM-DD]	Reproduced and confirmed
[YYYY-MM-DD]	CVE id requested
[YYYY-MM-DD]	Fix released in [A.B.C]
[YYYY-MM-DD]	Advisory published

References

- [Release notes / tag]
- [Commit, if the repository is public]
- [CVE record]
- [Any upstream or related advisory]

Contact

[security@yourdomain.example]

After publication

Link it from your release notes - plenty of people read those and never see advisories. Use your announcement list; a published page is not a notification. If severity is high and you know who runs affected versions, email them. Add the reporter to the acknowledgements page your security.txt points at. Update the advisory if facts change, moving the “last updated” date. Record the outcome in your decision log (file 05).

05 - The Five Files

NOT LEGAL ADVICE. A checklist and templates. Not legal advice, not a certification, not a compliance determination. The support and end-of-life statement can interact with your customer contracts - see the note there.

The list

If all five exist, are current, and are where people expect them, you have the hygiene that regulators and enterprise buyers assume you already had.

#	File	Where	Why
1	SECURITY.md	repo root or .github/	GitHub surfaces it automatically; first place a developer looks
2	security.txt	/.well-known/security.txt	Where a researcher's tooling looks. RFC 9116
3	Disclosure policy	a stable public URL	The commitments behind the two files above
4	Decision log	private, in the repo	The only record of <i>why</i> you did what you did
5	Support / EOL statement	a stable public URL	Makes "unsupported" a published fact, not an argument

Files 2 and 3 are covered in files 01 and 02.

```
[ ] SECURITY.md exists and names a working contact
[ ] security.txt is served, and Expires is in the future
[ ] The disclosure policy URL in security.txt actually loads
[ ] The decision log exists and has at least one dated entry
[ ] The support statement names specific versions and dates
[ ] Two named humans read the security inbox
[ ] Someone outside the company sent a test report and got a reply
```

The last line is the only one that tests the whole chain. Do it.

1. SECURITY.md

Short on purpose. It points at the real policy rather than duplicating it - two copies means one is stale and you will not know which.

```
# Security Policy

## Reporting a vulnerability

Please report security issues to [security@yourdomain.example].

Do not open a public issue for a security vulnerability.

[If your repo is public and you have enabled private reporting:]
You can also use GitHub's private vulnerability reporting: go to
the Security tab and click Report a vulnerability.

We will acknowledge your report within [2] business days and
give you an initial assessment within [10] business days.

Our full disclosure policy, including scope and safe harbour, is at
[https://yourdomain.example/security/disclosure-policy].

## Supported versions

| Version | Supported |
|-----|-----|
| [2.5.x] | Yes      |
| [2.4.x] | Security fixes only, until [YYYY-MM-DD] |
| [< 2.4] | No       |

Full support policy: [https://yourdomain.example/support-policy]

## Our advisories

Published security advisories: [https://yourdomain.example/security/advisories]
```

Keep these numbers identical to the ones in your disclosure policy. Two different promises in two places is worse than none.

2 and 3. security.txt and the disclosure policy

Covered in files 01 and 02. Two things to carry forward: set a calendar reminder for the

Expires date now, or put the check in CI, because an expired file is worse than none; and publish the policy at a URL that will not move, since both other files point at it.

4. The dated decision log

Why people skip it

It has no immediate user. Nobody asks for it. It is not in a security questionnaire or a repo template. Nothing breaks the week you do not write it.

And it is the only file here that cannot be reconstructed later. You can write a `security.txt` in ten minutes at any future point, or adopt a disclosure policy the day before a customer asks. You cannot go back to March and recover why you decided a report was not exploitable, who agreed, or what evidence you looked at. That has a half-life of about one person's memory of a busy afternoon, then it is gone.

Why it matters later

An enterprise security review. A prospect asks how you handle vulnerability reports. With a log you answer in one message with real examples. Without one you write a page of process description that a reviewer correctly reads as aspirational.

A report comes back. Eighteen months on the same issue resurfaces, now clearly exploitable because something else changed. With a log you see immediately what your reasoning was and which assumption broke. Without one you start from zero and will probably reach the same wrong answer.

Someone asks what you knew. Customer, insurer, regulator, or opposing counsel - the question is identical: what did you know, when, and what did you do. A dated note written *before* you knew how the story ended is worth far more than a reconstruction written after. A reconstruction is an argument; a contemporaneous note is evidence.

Both EU regimes in the README lean on exactly this. The CRA's obligations attach to what you knew about active exploitation and when; the PLD's expectations likewise turn on what a manufacturer knew and did. The specific evidence practices each calls for are in the respective kits. The underlying habit - write the decision and the date at the moment you make it - is what both assume you already have. Three minutes per entry.

The rule

Write the entry when you decide, not when you remember to. An entry written three weeks later is a reconstruction. And if you only log the interesting cases you have a highlight reel - the boring “not exploitable, here is why” entries are the ones that turn out to matter.

Keep it in the repository so it is version-controlled and timestamped by git, in a private repo or private directory (docs/security/decision-log.md). Markdown is enough. Not a wiki page anyone can silently edit, and not a document with no revision history. Keep it separate from triage worksheets: those are per-report working documents, the log is the durable record, and it should stay short enough that someone will read it.

```
# Security Decision Log
```

```
Append-only. Add entries at the bottom. Never edit or delete a past
entry - if something was wrong, add a correcting entry that says so
and references the original.
```

```
Format: one entry per decision. Keep each under ten lines.
```

```
---
```

```
## [YYYY-MM-DD] SEC-2026-001 - [One-line description]
```

```
**Decision:** [ fix now / fix in next release / backlog /
not a vulnerability / out of scope / forwarded upstream ]
```

```
**Decided by:** [NAME]
```

```
**Present / consulted:** [NAMES]
```

```
**What we knew:** [Two or three lines. The facts as they stood
today, not as they turned out. Include what you did not know.]
```

```
**Why:** [The reasoning. If you concluded it was not exploitable,
state the specific reason - "the endpoint requires an
authenticated admin session and the vulnerable parameter is
validated at the router" - not "assessed as low risk".]
```

```
**Active exploitation:** [ no evidence - basis: e.g. "app logs
searched 2026-09-15, 90-day retention, no matches" / evidence
found - detail / not checked - why not ]
```

```
**Follow-up:** [Action, owner, date. Or "none".]
```

```
---
```

What else belongs in it

Not only vulnerability reports. Anything where you made a judgement a future reader would want the reasoning for: a decision **not** to fix something (the most valuable entries in any log); dropping support for a version; accepting a known risk, with its compensating control and review date; a dependency CVE you assessed as not reaching your code, with the reason; a change to your response commitments; a decision about whether to notify customers, either way; the date you last tested that the security inbox works.

A worked entry

```
## 2026-03-04 SEC-2026-002 - Path traversal report in export endpoint

**Decision:** Not a vulnerability in shipped product.
**Decided by:** P. Raman
**Present / consulted:** J. Okonkwo (reviewed the fix path)

**What we knew:** Reporter demonstrated traversal against
/api/export on a local build with DEV_ALLOW_RAW_PATHS=1. That flag
is set only in docker-compose.dev.yml. Confirmed it is not set in
any released artefact; checked the 2.4 and 2.5 release images.

**Why:** The vulnerable branch is unreachable without the dev flag.
No released configuration enables it. We are, separately, removing
the flag entirely in 2.6 because it is a foot-gun.

**Active exploitation:** No evidence. Not applicable - path is not
reachable in production builds.

**Follow-up:** Remove DEV_ALLOW_RAW_PATHS in 2.6 - J. Okonkwo -
2026-04-30. Replied to reporter 2026-03-04 with this reasoning and
offered credit; they declined.
```

Note what it does: records the specific technical reason, says what was checked and when, and stays understandable to someone who was not there. That is the whole bar.

5. Support and end-of-life statement

Without it, every decision about an old version is an argument. With it, “2.3 stopped receiving security fixes on 30 June” is a fact you published in advance. It also lets your disclosure policy put old versions out of scope without that looking like an excuse

invented after the report arrived.

```
# Support and End-of-Life Policy

**Last updated: [YYYY-MM-DD]**

## What we support

| Version | Released | Full support until | Security fixes until |
|-----|-----|-----|-----|
| [2.5.x] | [2026-05-01] | [2027-05-01] | [2027-11-01] |
| [2.4.x] | [2025-11-01] | [2026-05-01] | [2026-11-01] |
| [2.3.x] | [2025-04-01] | ended [2025-11-01] | ended [2026-04-01] |

**Full support** means bug fixes, security fixes, and compatibility updates.

**Security fixes only** means we will fix vulnerabilities we assess as [high severity or above], and nothing else.

**Ended** means we will not ship fixes for that version. We may still describe it in an advisory so you know whether you are affected.

## How long versions are supported

[Describe your rule in one or two sentences. For example: "Each minor release receives full support for 12 months from its release date, and security fixes for a further 6 months." Or: "We support the current release and the one before it."]

>> A rule people can plan against is worth more than a table you
>> update ad hoc. Publish the rule and let the table follow from it.

## Notice before end of life

We will announce the end-of-life date for a version at least
[90] days in advance, at [https://yourdomain.example/changelog]
and by email to [customers on a support plan / the announce list].

## Upgrading

[Link to upgrade guide, migration notes, or LTS options.]

## Questions

[support@yourdomain.example]
```

Two dates per version, not one. End of full support and end of security fixes are

different events. Collapsing them means either committing to feature work longer than you want, or cutting off security fixes sooner than customers expect.

A rule beats a table. Publish the rule (“12 months plus 6”) and derive the table from it, so nobody has to ask what happens to a version you have not announced yet.

Check your contracts first. If you have signed enterprise agreements, MSAs, or support addenda with specific commitments, your public EOL statement must not undercut them. This is the one place in this file where a look from whoever handles your contracts is genuinely worth it - a public page contradicting a signed agreement is a bad thing to find during a renewal.

Machine-readable EOL data. endoflife.date maintains a public catalogue with a free API at <https://endoflife.date/api/v1/products> (473 products, no key). It is also useful for checking the EOL status of things *you* depend on. Whether to submit your own product is your call; it is a community-maintained dataset, not an official register.

Closing

You are finished when all five files exist, the dates in them are real, two people read the inbox, and someone outside the company has sent a test report and had a reply from a human.

That is the whole hygiene layer. It is not large. Most teams do not have it, and the reason is not difficulty - it is that no single week is the week it becomes urgent, right up until the week it does.

If the EU Cyber Resilience Act (Regulation (EU) 2024/2847, Article 14 reporting enforceable from 11 September 2026 - <https://eur-lex.europa.eu/eli/reg/2024/2847/oj>) or the Product Liability Directive (Directive (EU) 2024/2853, applying to products placed on the market after 8 December 2026 per Corrigendum 2026/90364 - <http://data.europa.eu/eli/dir/2024/2853/corrigendum/2026-05-07/oj>) applies to you, this pack is the floor, not the ceiling. The reporting mechanics and the liability-side evidence practices are separate, substantial bodies of work, covered in the respective kits.

If neither applies, you are done. Put a reminder in the calendar to re-read these five files in twelve months.